

Topic 3: Streaming Wavelet Data

Contents

- [layout.csl](#)
- [pe_program.csl](#)
- [run.py](#)
- [commands.sh](#)

Often, CSL programs contain tasks that are activated in response to the arrival of wavelets of specific colors. Such tasks are also called Wavelet-Triggered Tasks, or data tasks.

In this example, the `comptime` block binds a data task to a `data_task_id` created from a `memcpy` streaming color, which receives data from the host. The routing of the color `MEMCPYH2D_DATA_1` must not be defined. The `memcpy` module will figure out the routing of `MEMCPYH2D_DATA_1`.

Given the task and color association and the route, when a wavelet of color `MEMCPYH2D_DATA_1` arrives at the router, it is forwarded to the CE, which then activates `main_task`. The wavelet's payload field is received in the argument to the task, and the code uses the wavelet data to update a global variable.

layout.csl

```

// Color map/ WSE-2 task ID map
// On WSE-2, data tasks are bound to colors (IDs 0 through 24)
//
// ID var          ID var  ID var          ID var
// 0 MEMCPY_H2D_DATA_1    9      18      27 reserved (memcpy)
// 1 MEMCPY_D2H_DATA_1    10     19      28 reserved (memcpy)
// 2                   11     20      29 reserved
// 3                   12     21 reserved (memcpy) 30 reserved (memcpy)
// 4                   13     22 reserved (memcpy) 31 reserved
// 5                   14     23 reserved (memcpy) 32
// 6                   15     24                33
// 7                   16     25                34
// 8                   17     26                35

// WSE-3 task ID map
// On WSE-3, data tasks are bound to input queues (IDs 0 through 7)
//
// ID var          ID var  ID var          ID var
// 0 reserved (memcpy)    9      18      27 reserved (memcpy)
// 1 reserved (memcpy)    10     19      28 reserved (memcpy)
// 2 main_task_id        11     20      29 reserved
// 3                   12     21 reserved (memcpy) 30 reserved (memcpy)
// 4                   13     22 reserved (memcpy) 31 reserved
// 5                   14     23 reserved (memcpy) 32
// 6                   15     24                33
// 7                   16     25                34
// 8                   17     26                35

// IDs for memcpy streaming colors
param MEMCPYH2D_DATA_1_ID: i16;
param MEMCPYD2H_DATA_1_ID: i16;

const memcpy = @import_module("<memcpy/get_params>", .{
    .width = 1,
    .height = 1,
    .MEMCPYH2D_1 = MEMCPYH2D_DATA_1_ID,
    .MEMCPYD2H_1 = MEMCPYD2H_DATA_1_ID
});

layout {
    @set_rectangle(1, 1);

    @set_tile_code(0, 0, "pe_program.csl", .{
        .memcpy_params = memcpy.get_params(0),
    });
}

```

pe_program.csl

```

// Not a complete program; the top-level source file is layout.csl.

param memcpy_params;

const sys_mod = @import_module("<memcpy/memcpy>", memcpy_params);

// Queue IDs
const h2d_data_1_iq: input_queue = @get_input_queue(2);
const d2h_data_1_oq: output_queue = @get_output_queue(3);

// Data task main_task triggered by wltS along MEMCPYH2D_DATA_1
// On WSE-2, data task IDs are created from colors; on WSE-3, from input queues
const main_task_id: data_task_id =
  if (@is_arch("wse2")) @get_data_task_id(@get_color(@bitcast(u16, sys_mod.MEMCPYH2D_1)))
  else if (@is_arch("wse3")) @get_data_task_id(h2d_data_1_iq);

export var global: i16 = 0;

const out_dsd = @get_dsd(fabout_dsd, if (@is_arch("wse3")) .{
  .extent = 1,
  .output_queue = d2h_data_1_oq
} else .{
  .extent = 1,
  .fabric_color = @get_color(@bitcast(u16, sys_mod.MEMCPYD2H_1)),
  .output_queue = d2h_data_1_oq
});

task main_task(wavelet_data: i16) void {
  global = wavelet_data;
  // The non-async operation works here because only one wavelet is sent
  // It would be better to use async operation with .{async = true}
  @mov16(out_dsd, global);
}

comptime {
  @bind_data_task(main_task, main_task_id);

  // On WSE-3, we must explicitly initialize input and output queues
  if (@is_arch("wse3")) {
    @initialize_queue(h2d_data_1_iq, .{ .color = @get_color(@bitcast(u16, sys_mod.MEMCPYH2D_1))
  });
    @initialize_queue(d2h_data_1_oq, .{ .color = @get_color(@bitcast(u16, sys_mod.MEMCPYD2H_1))
  });
  }
}

```

run.py

```

#!/usr/bin/env cs_python

import argparse

```

```

import json
import numpy as np

from cerebras.sdk.sdk_utils import memcopy_view, input_array_to_u32
from cerebras.sdk.runtime.sdkruntimepybind import SdkRuntime, MemcpyDataType # pylint:
disable=no-name-in-module
from cerebras.sdk.runtime.sdkruntimepybind import MemcpyOrder # pylint: disable=no-name-in-
module

parser = argparse.ArgumentParser()
parser.add_argument('--name', help='the test name')
parser.add_argument("--cmaddr", help="IP:port for CS system")
args = parser.parse_args()
dirname = args.name

# Parse the compile metadata
with open(f"{dirname}/out.json", encoding="utf-8") as json_file:
    compile_data = json.load(json_file)
    params = compile_data["params"]
    MEMCPYH2D_DATA_1 = int(params["MEMCPYH2D_DATA_1_ID"])
    MEMCPYD2H_DATA_1 = int(params["MEMCPYD2H_DATA_1_ID"])
    print(f"MEMCPYH2D_DATA_1 = {MEMCPYH2D_DATA_1}")
    print(f"MEMCPYD2H_DATA_1 = {MEMCPYD2H_DATA_1}")

memcpy_dtype = MemcpyDataType.MEMCPY_16BIT
runner = SdkRuntime(dirname, cmaddr=args.cmaddr)

runner.load()
runner.run()

input_tensor = np.array([42], dtype=np.int16)

print("step 1: streaming H2D")
# "input_tensor" is a 1d array
# The type of input_tensor is int16, we need to extend it to uint32
# There are two kind of extension when using the utility function input_array_to_u32
#   input_array_to_u32(np_arr: np.ndarray, sentinel: Optional[int], fast_dim_sz: int)
# 1) zero extension:
#   sentinel = None
# 2) upper 16-bit is the index of the array:
#   sentinel is Not None
#
# In this example, the upper 16-bit is don't care because pe_program.csl only define
# WTT to read lower 16-bit
#tensors_u32 = runtime_utils.input_array_to_u32(input_tensor, 1, 1)
tensors_u32 = input_array_to_u32(input_tensor, 1, 1)
runner.memcpy_h2d(MEMCPYH2D_DATA_1, tensors_u32, 0, 0, 1, 1, 1, \
    streaming=True, data_type=memcpy_dtype, order=MemcpyOrder.COL_MAJOR, nonblock=True)

print("step 2: streaming D2H")
# The D2H buffer must be of type u32
out_tensors_u32 = np.zeros(1, np.uint32)
runner.memcpy_d2h(out_tensors_u32, MEMCPYD2H_DATA_1, 0, 0, 1, 1, 1, \
    streaming=True, data_type=memcpy_dtype, order=MemcpyOrder.COL_MAJOR, nonblock=False)
# remove upper 16-bit of each u32

```

```
result_tensor = memcpy_view(out_tensors_u32, np.dtype(np.int16))

runner.stop()

# Ensure that the result matches our expectation
np.testing.assert_equal(result_tensor, [42])
print("SUCCESS!")
```

commands.sh

```
#!/usr/bin/env bash

set -e

cslc --arch=wse3 ./layout.csl --fabric-dims=8,3 \
--fabric-offsets=4,1 -o out \
--params=MEMCPYH2D_DATA_1_ID:0 \
--params=MEMCPYD2H_DATA_1_ID:1 \
--memcpy --channels=1 --width-west-buf=0 --width-east-buf=0
cs_python run.py --name out
```